

ONLINE COURSE ENROLLMENT SYSTEM

SCALABLE
ARCHITECTURE

HIGH
PERFORMANCE

MySQL

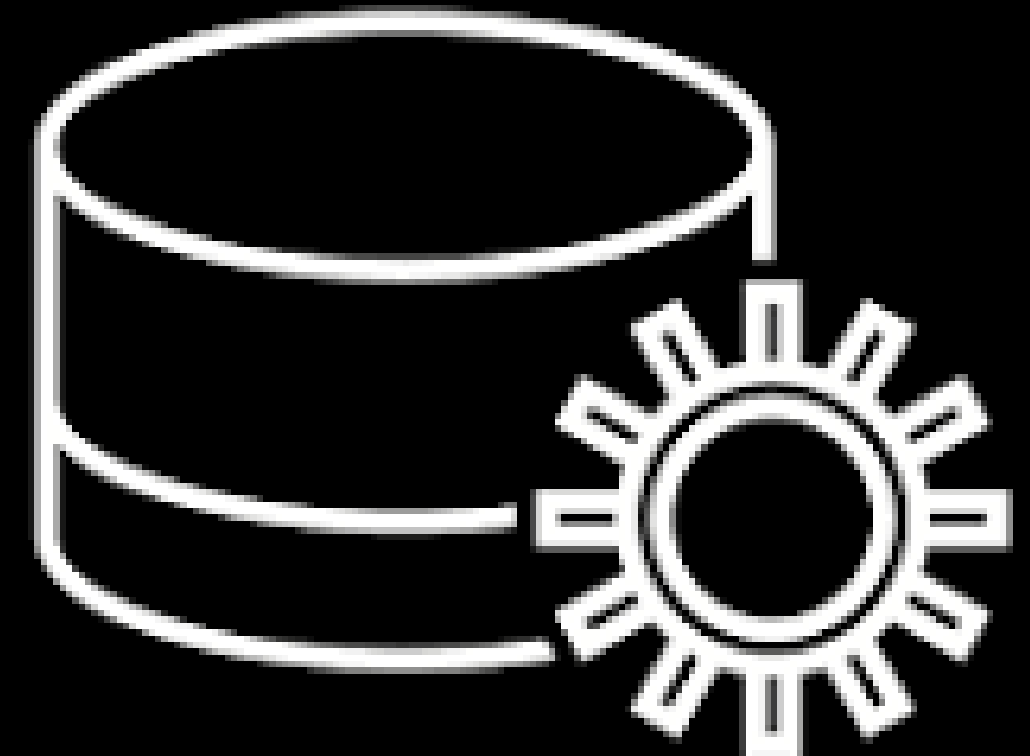
OPEN SOURCE

GLOBAL REACH



OVERVIEW OF THE PROJECT

- This project is a SQL-based Online Course Enrollment System designed to manage students, courses, enrollments, and feedback.
- The system helps track student participation, course performance, and feedback trends.
- It uses relational database design and advanced SQL queries to generate meaningful business insights.
- Key analysis includes course completion rate, top-rated courses, and student engagement.



WHY MYSQL FOR THIS PROJECT?



- MySQL is used because it is a reliable relational database system that efficiently manages structured data like students, courses, and enrollments. It supports complex queries, relationships, and data analysis, making it suitable for this project.

DATABASE DESIGN

- The database is designed using a relational model with four main entities: Students, Courses, Enrollments, and Feedbacks.
- Enrollments acts as a bridge table connecting students and courses, while Feedbacks stores course reviews given by students.
- Primary and foreign keys are used to maintain data integrity and relationships.

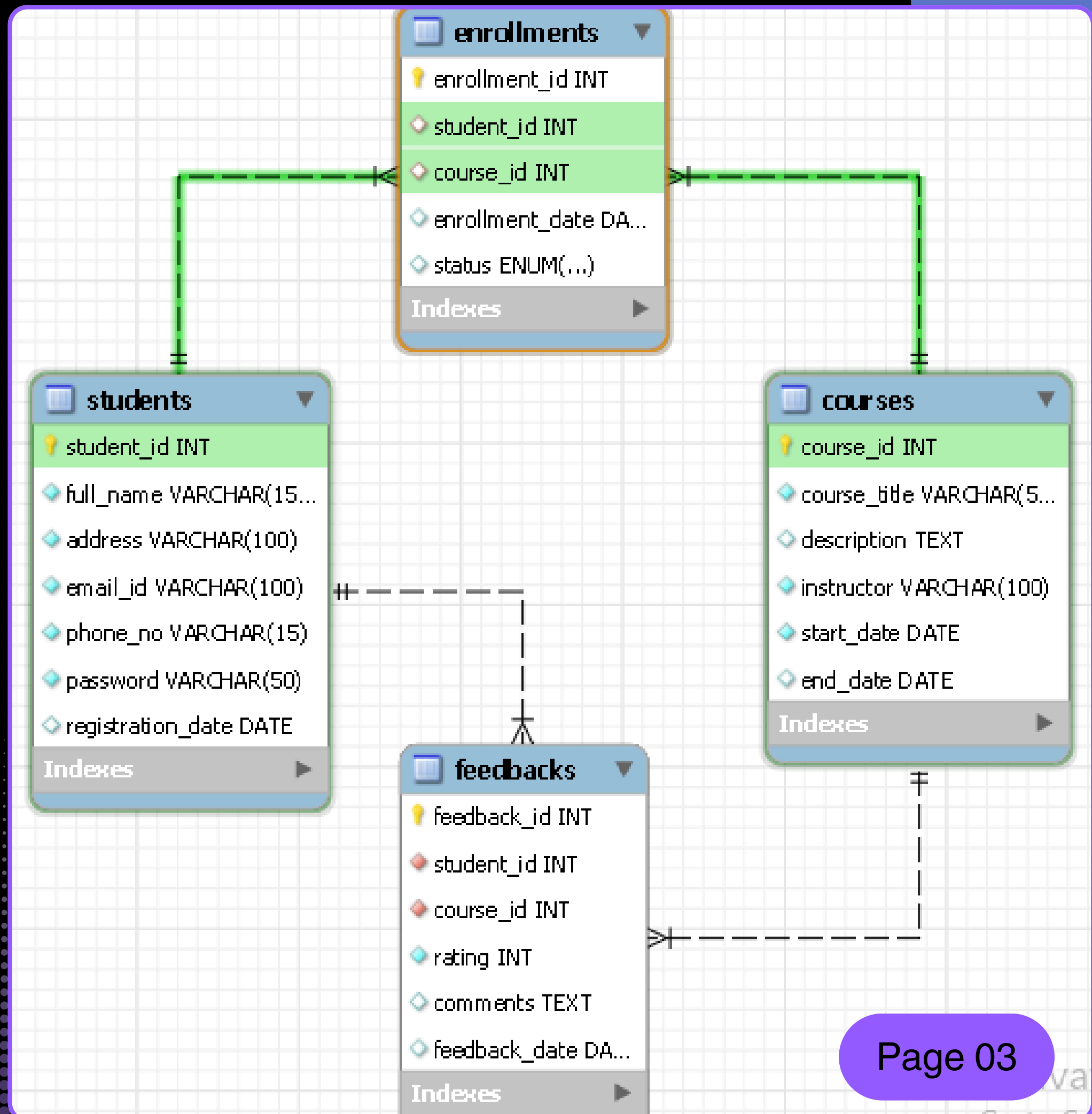


TABLE STRUCTURE

Students Table

```
# Students Table
create table students (
student_id int auto_increment primary key,
full_name varchar(150) not null,
address varchar(100) not null,
email_id varchar(100) unique not null,
phone_no varchar(15) unique not null,
password varchar(50) not null,
registration_date date default(current_date));
```

- Stores student personal details and login information
- Each student has a unique ID, email, and phone number

Courses Table

```
# Courses Table
create table courses (
course_id int auto_increment primary key,
course_title varchar(50) not null,
description text,
instructor varchar(100) not null,
start_date date not null,
end_date date
);
```

- Contains course details like title, instructor, and duration
- Each course has a unique course ID

TABLE STRUCTURE

Enrollments Table

```
# Enrollment Table
create table enrollments (
  enrollment_id int auto_increment primary key,
  student_id int,
  course_id int,
  enrollment_date date default(current_date),
  status enum('Enrolled', 'Completed', 'Cancelled',
  'Not Specified') default 'Not Specified',
  foreign key (student_id)
  references students(student_id),
  foreign key (course_id)
  references courses(course_id)
);
```

- Links students with courses (many-to-many relationship)
- Tracks enrollment date and course status

Feedbacks Table

```
# Feedback Table
create table feedbacks (
  feedback_id int auto_increment primary key,
  student_id int not null,
  course_id int not null,
  rating int check (rating >= 1 AND rating <= 5) not null,
  comments text,
  feedback_date date default(current_date),
  foreign key (student_id) references students(student_id)
  foreign key (course_id) references courses(course_id)
);
```

- Stores ratings and comments given by students
- Helps analyze course performance and satisfaction

1. Find the Student id, names & phone no of students who have given a rating above 4 in any course. (Sub query)

Query

```
select student_id, full_name, phone_no
from students
where student_id in (
select student_id from feedbacks where rating > 4);
```

Output

	student_id	full_name	phone_no
▶	1	Rahul Sharma	9876543210
	3	Amit Das	9876543212
●	NULL	NULL	NULL

2. Find the number of days each course runs. (DT)

Query

```
select course_id, Course_Title,  
DATEDIFF(COALESCE(end_date, CURRENT_DATE), start_date) As duration_days  
from courses  
where start_date is not null;
```

Output

	course_id	Course_Title	duration_days
▶	1	SQL Basics	31
	2	Advanced SQL	28
	3	Web Development	411
	4	Python Programming	59
	5	Data Analytics	439

3. Find all students who have enrolled but not given feedback. (CTE + Anti join)

Query

```
with enrolled_students as (  
  select e.student_id, e.course_id from enrollments e)  
  select s.student_id, s.full_name, c.course_title from enrolled_students es  
  join students s on s.student_id = es.student_id  
  join courses c on c.course_id = es.course_id  
  left join feedbacks f on f.student_id = es.student_id  
  and f.course_id = es.course_id where f.feedback_id is null;
```

Output

	student_id	full_name	course_title
▶	4	Sneha Roy	SQL Basics
	1	Rahul Sharma	Advanced SQL
	3	Amit Das	Advanced SQL
	2	Priya Singh	Web Development
	5	Arjun Verma	Web Development

4. Find students who have enrolled in more than or equal 2 courses. (CTE+ Having)

Query

```
with student_course_count as
(select e.student_id, count(distinct(e.course_id)) as course_count
from enrollments e
group by e.student_id
having course_count >= 2)
select s.student_id, s.full_name, sc.course_count
from student_course_count sc
join students s ON s.student_id = sc.student_id;
```

Output

	student_id	full_name	course_count
▶	1	Rahul Sharma	2
	2	Priya Singh	2
	3	Amit Das	2
	4	Sneha Roy	2
	5	Arjun Verma	2

5. Find top 3 courses with highest average rating. (CTE+ Window Dense_Rank)

Query

```
with course_avg_rating as
(select c.course_id,c.course_title, avg(f.rating) as avg_rating
from courses c
join feedbacks f
on c.course_id = f.course_id
group by c.course_id, c.course_title),
course_rank as (
select course_id, course_title, avg_rating, dense_rank()
over(order by avg_rating desc) as rank_no
from course_avg_rating)
select course_id, course_title, avg_rating
from course_rank
where rank_no <= 3
order by avg_rating desc;
```

Output

	course_id	course_title	avg_rating
	1	SQL Basics	4.5000
	4	Python Programming	4.5000
	5	Data Analytics	3.0000

6. Find courses that started in the current year and have more than 5 enrolments.(current month with Data Filter + Having)

Query

```
select c.course_id, c.course_title,  
count(e.enrollment_id) as enrollment_count  
from courses c  
join enrollments e  
on c.course_id = e.course_id  
where year(c.start_date) = year(curdate())  
group by course_id, course_title  
having enrollment_count >= 1  
order by enrollment_count desc;
```

Output

	course_id	course_title	enrollment_count
	1	SQL Basics	3
	2	Advanced SQL	2

7. Write a stored procedure to get all feedback for a given course.(Store Procedure)

Query

```
delimiter //  
create procedure get_feedbacks(in p_course_id int, in p_min_rating int)  
begin  
    select s.full_name, f.rating, f.comments, f.feedback_date  
    from students s join feedbacks f  
    on s.student_id = f.feedback_id  
    where s.student_id = p_course_id  
    and f.rating >= p_min_rating  
    order by f.feedback_date desc;  
end //  
delimiter ;  
  
call get_feedbacks(2, 0);
```

Output

full_name	rating	comments	feedback_date
Priya Singh	4	Very good	2026-04-15

8. Get the list of students, their course titles, and feedback ratings. (include missing Ratings) (Joins)

Query

```
select s.full_name, c.course_title, e.enrollment_date, f.rating as feedback_rating
from enrollments e
join students s
on e.student_id = s.student_id
join courses c
on e.course_id = c.course_id
left join feedbacks f
on e.student_id = f.student_id
and e.course_id = f.course_id;
```

Output

	full_name	course_title	enrollment_date	feedback_rating
▶	Rahul Sharma	SQL Basics	2026-04-15	5
	Rahul Sharma	Advanced SQL	2026-04-15	NULL
	Priya Singh	SQL Basics	2026-04-15	4
	Priya Singh	Web Development	2026-04-15	NULL
	Amit Das	Advanced SQL	2026-04-15	NULL
	Amit Das	Python Programming	2026-04-15	5
	Sneha Roy	SQL Basics	2026-04-15	NULL
	Sneha Roy	Data Analytics	2026-04-15	3
	Arjun Verma	Web Development	2026-04-15	NULL
	Ariun Verma	Pythn Programming	2026-04-15	4

9. Course completion rate (Aggregate + CASE)

Query

```
select c.course_id, c.course_title,  
concat(round(sum(case when e.status = "Completed" then 1 else 0 end) / count(*) * 100, 2), "%")  
as course_completion_rate from courses c  
join enrollments e  
on c.course_id = e.course_id  
group by c.course_id, c.course_title;
```

Output

	course_id	course_title	course_completion_rate
▶	1	SQL Basics	66.67%
	2	Advanced SQL	0.00%
	3	Web Development	0.00%
	4	Python Programming	100.00%
	5	Data Analytics	100.00%

10. Recent feedback trend (Date filter + aggregate)

Query

```
select c.course_id, c.course_title,  
count(f.feedback_id) as feedback_count,  
round(avg(f.rating), 2) as avg_rating  
from feedbacks f  
join courses c  
on f.course_id = c.course_id  
where f.feedback_date >= curdate() - interval 90 day  
group by c.course_id, c.course_title  
order by avg_rating desc;
```

Output

	course_id	course_title	feedback_count	avg_rating
▶	1	SQL Basics	2	4.50
	4	Python Programming	2	4.50
	5	Data Analytics	1	3.00

Thank You

This project successfully demonstrates the design and implementation of a relational database system for an online course platform.

Advanced SQL techniques were used to analyze student behavior, course performance, and feedback trends.

The project highlights practical problem-solving using real-world business scenarios.

